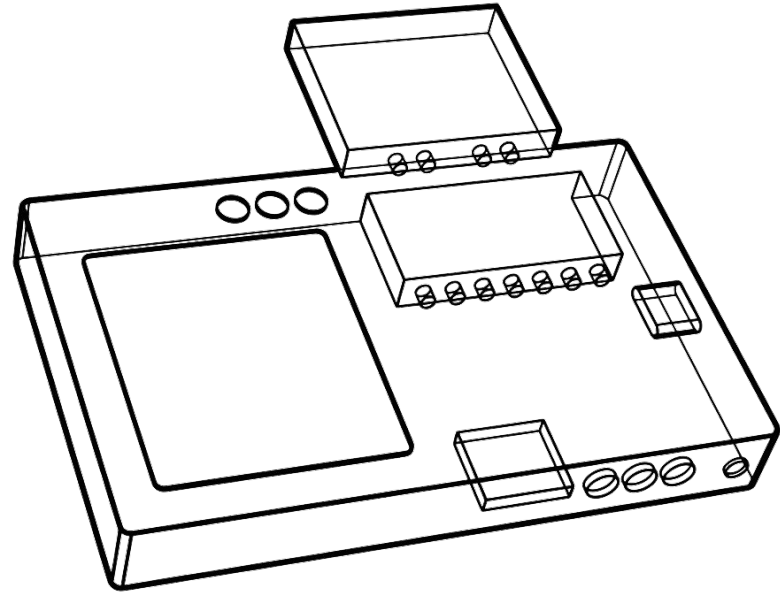


Diseño e implementación de un dispositivo IoT modular

Alejandro Rodríguez Mederos

Grado en Ingeniería Informática
ESIT · Universidad de La Laguna
La Laguna, junio 2026

Tutores: D. Iván Castilla · D. Santiago Torres



Estructura de la defensa

01 Motivación y objetivos

03 DSL: lenguaje de dominio

05 Prototipo y demostración

07 Conclusiones (EN)

02 Arquitectura del sistema

04 Servidor HTTP embebido

06 Gestión energética

08 Bibliografía

¿Por qué este dispositivo?

El problema

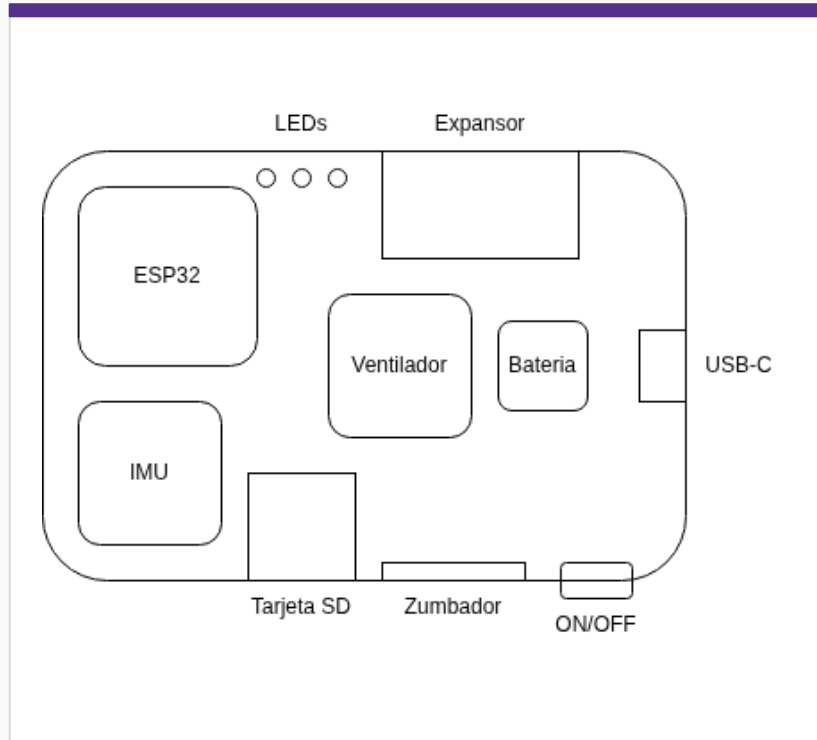
- Soluciones IoT altamente especializadas y cerradas
- El hardware no es reutilizable entre aplicaciones
- Adaptar el comportamiento exige recompilar el firmware
- Dependencia de servicios cloud para funcionar
- El usuario debe adaptarse al dispositivo, no al revés

La propuesta

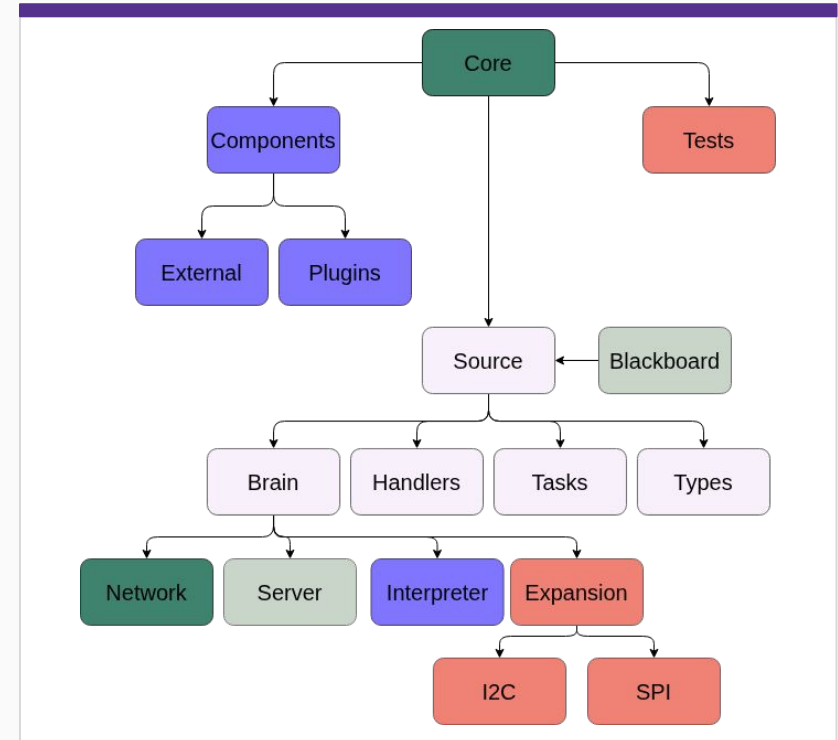
- Plataforma base reutilizable para múltiples aplicaciones
- Ampliación rápida mediante módulos externos
- DSL: programar sin herramientas externas ni recompilación
- Servidor web embebido, configuración desde el ordenador
- El dispositivo se adapta al usuario

Arquitectura del sistema

Hardware



Software



Programar el hardware sin recompilar

Lexer + Interpreter

Tokenización y traducción a C++

Control de flujo

Variables, condicionales, bucles, operadores

Comunicación I2C

Lectura y escritura de registros.

Logs y archivos

Registros en microSD.

temperatura.str – ejemplo real del prototipo

```
FILE "temp.log"
```

```
I2C INIT
```

```
I2C WRITE 0x76 0xF4 0x27
```

```
WAIT 1
```

```
I2C READLE 0x76 0x88 2 -> dig_T1
```

```
I2C READLE 0x76 0x8A 2 -> dig_T2
```

```
I2C READLE 0x76 0x8C 2 -> dig_T3
```

```
I2C READ 0x76 0xFA 1 -> temp_msb
```

```
I2C READ 0x76 0xFB 1 -> temp_lsb
```

```
I2C READ 0x76 0xFC 1 -> temp_xlsb
```

```
temp_raw = (temp_msb << 12) | (temp_lsb << 4) | (temp_xlsb >> 4)
```

```
var1 = ((temp_raw / 8) - (dig_T1 * 2)) * dig_T2 / 2048
```

```
var2 = (((temp_raw / 16) - dig_T1) * ((temp_raw / 16) - dig_T1) / 4096) *  
dig_T3 / 16384 21
```

```
t_fine = var1 + var2
```

```
temp = (t_fine * 5 / 256) / 100
```

```
PRINT "Temperatura (grados C):" temp
```

Gestión desde el navegador

Modo Station (STA)

Se conecta a una red existente.

Ideal para entornos fijos.

Acceso desde toda la red local.

Modo Access Point (AP)

El dispositivo crea su propia red Wi-Fi.

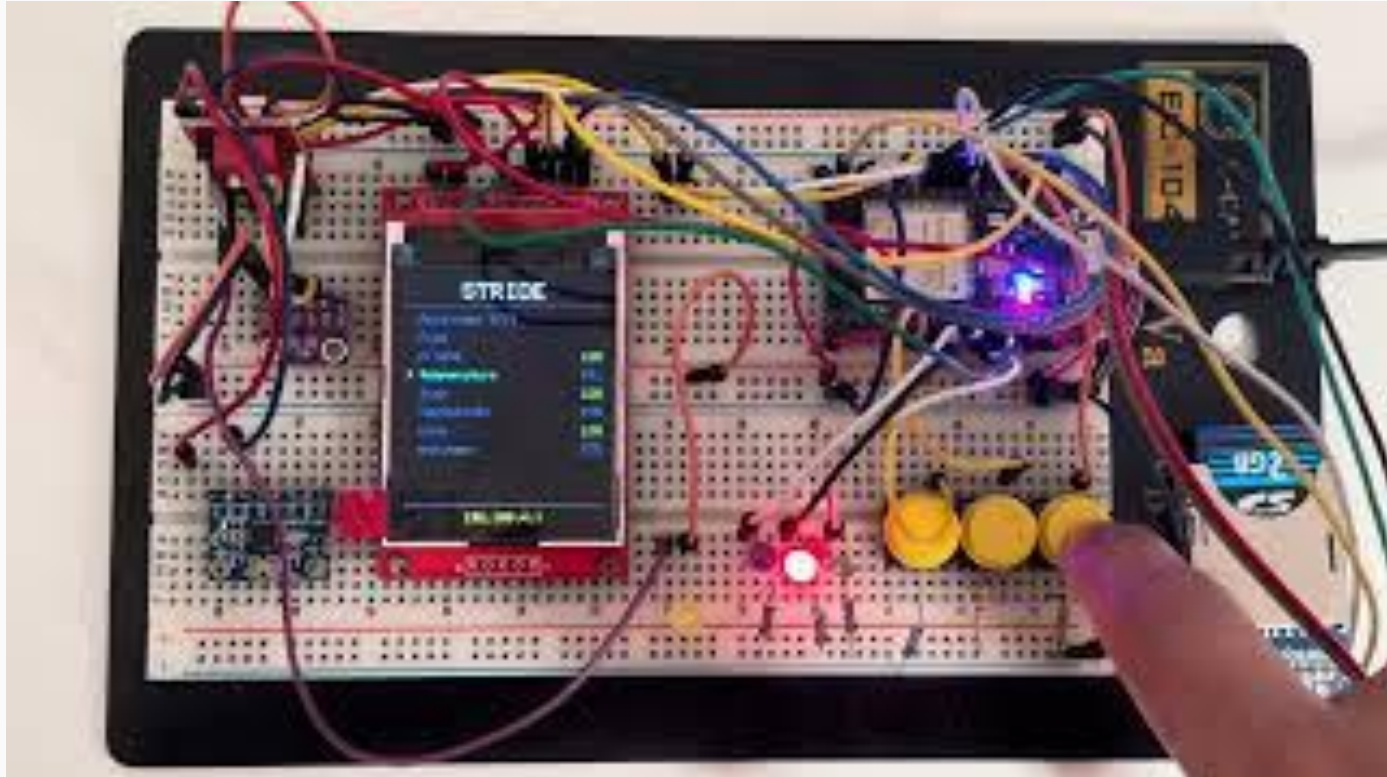
Uso portátil y autónomo.

Acceso directo desde cualquier dispositivo.

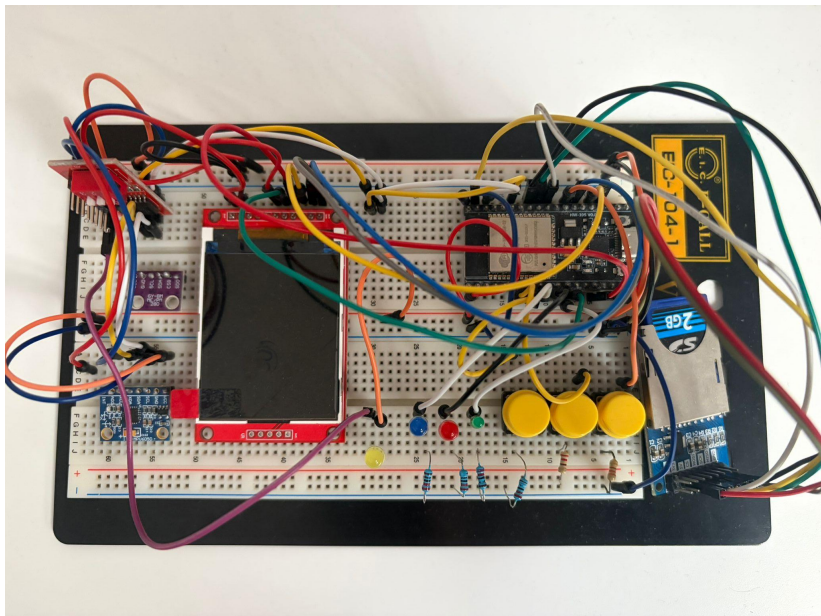
The screenshot displays the 'Stride' web interface. On the left is a dark sidebar with navigation links: 'Browser', 'Referencia', and 'Ajustes'. The main content area has a top status bar indicating 'Ejecutando: movimiento.str'. Below this, the 'Archivos SD' section shows '6 archivos' and a '+ Nuevo' button. A table lists the files with columns for 'NOMBRE' and 'ACCIONES'.

NOMBRE	ACCIONES
prints.log	[Icono de documento] [Icono de ojo] [Icono de papelera]
temperatura.str	[Icono de documento] [Icono de play] [Icono de ojo] [Icono de papelera]
temp.log	[Icono de documento] [Icono de ojo] [Icono de papelera]
movimiento.str	[Icono de documento] [Icono de play] [Icono de ojo] [Icono de papelera]
move.log	[Icono de documento] [Icono de ojo] [Icono de papelera]
expansor.str	[Icono de documento] [Icono de play] [Icono de ojo] [Icono de papelera]

Demostración del sistema en funcionamiento



Del diseño conceptual al hardware real



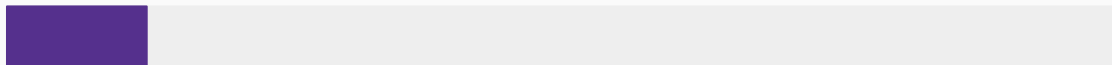
Funcionalidades validadas

- Ejecución de programas DSL en tiempo real
- Sensores temperatura y unidad de medida inercial vía I2C
- Dashboard web accesible desde navegador
- Logs con timestamp
- Navegación por menú
- Modos AP y STA configurables
- Pruebas automatizadas del núcleo

Autonomía estimada del sistema

Reposo

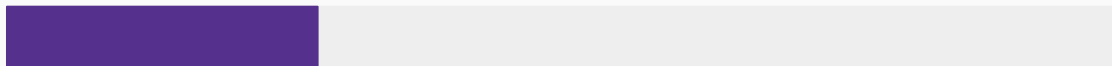
Pantalla atenuada,
ESP32 en bajo consumo



~45 mA
~190h

Modo STA

Conectado a
red Wi-Fi existente



~155 mA
~58h

Modo AP

Generando su
propia red Wi-Fi



~240 mA
~37h

Uso intensivo

Wi-Fi activo y
accesos frecuentes a SD



~385 mA
~23h

Valores calculados a partir de las especificaciones de los componentes. No medidos físicamente en el prototipo actual.

Conclusions & Future Work

Conclusions

- All initial objectives successfully achieved
- Modular, reusable IoT platform validated across multiple scenarios
- DSL proved effective, hardware interaction without recompilation
- Layered architecture ensures maintainability and scalability
- Energy analysis confirms viability for portable battery-powered use
- Prototype validates architecture, DSL and embedded server end-to-end

Future Work

- **DSL**
 - Event-driven execution
 - User-defined subroutines
 - SPI support + new data types
 - Implementation of a parser
- **Hardware**
 - Custom PCB board
 - Full power & thermal validation
- **Software**
 - Using the tool ESP-Claw

Bibliografía

[1] - [12]

[1] Luigi Atzori, Antonio Iera y Giacomo Morabito. «The Internet of Things: A Survey». Computer Networks 54.15 (2010). doi: 10.1016/j.comnet.2010.05.010.

[2] Weisong Shi et al. «Edge Computing: Vision and Challenges». IEEE Internet of Things Journal 3.5 (2016). doi: 10.1109/JIOT.2016.2579198.

[3] Marjan Mernik, Jan Heering y Anthony M. Sloane. «When and How to Develop Domain-Specific Languages». ACM Computing Surveys 37.4 (2005). doi: 10.1145/1118890.1118892.

[4] «Industrial IoT Predictive Maintenance». ScienceDirect (2025).

[5] «Cost-Efficient Modular Sensor Platform». ScienceDirect (2025).

[6] «Smart Agriculture Soil and Crop Optimization». ScienceDirect (2025).

[7] «Modular Weather Station Based on Microservices» (2025).

[8] Espressif Systems. ESP32-S3 Wi-Fi + Bluetooth® LE SoC. 2024.

[9] Espressif Systems. ESP32-S3 Series Datasheet. v2.2. 2024.

[10] Arduino. Arduino UNO R4 WiFi User Manual. SKU: ABX00087. 2023.

[11] Texas Instruments. MSP430™ Ultra-Low-Power Microcontrollers. SLAS735J. 2013.

[12] Raspberry Pi Ltd. Raspberry Pi Hardware Documentation. 2024.

[13] - [24]

[13] Google LLC. Coral Technology: Edge TPU Overview. 2023.

[14] Espressif Systems. ESP-NOW — ESP-IDF Programming Guide. 2024.

[15] Espressif Systems. ESP-IDF Programming Guide. 2026.

[16] Espressif Systems. ESP-IDF Build System. 2026.

[17] Espressif Systems. ESP32 Arduino Core's Documentation. 2026.

[18] Refactoring.Guru. Design Patterns. 2026.

[19] Espressif Systems. Network API Reference (WiFi and HTTP Server). 2026.

[20] Espressif Systems. I2C Driver API Reference. 2026.

[21] Iovyan03. LovyanGFX. GitHub. 2026.

[22] Espressif Systems. FreeRTOS in ESP-IDF. 2026.

[23] Espressif Systems. ESP32-S3 Series Datasheet — Power Consumption Characteristics. v2.2. 2024.

[24] Espressif Systems. Chat Coding Edge AI Agent Framework for IoT (ESP-Claw). 2026.

Gracias

¿Preguntas?

ESP32 · DSL embebido · Servidor HTTP · Modular y portable